

Starta ett React-projekt med Vite & TypeScript

En guide till installation, folderstruktur och hur applikationen startar

Varför Vite? Tidigare användes verktyg som Create React App (CRA), men idag är **Vite** standard. Vite är extremt mycket snabbare på att starta upp utvecklingsmiljön och bygga projektet tack vare modern teknik som ES-moduler direkt i webbläsaren.

1. Skapa ett nytt projekt

För att skriva moderna React-applikationer kombinerar vi det nästan alltid med TypeScript. Följ dessa steg i din terminal för att skaffa projektet:

```
# 1. Kör Vite-installationskommandot (byt ut 'my-app' mot ditt projektnamn)
npm create vite@latest my-app -- --template react-ts

# 2. Gå in i mappen som skapades
cd my-app

# 3. Installera alla beroenden (node_modules)
npm install

# 4. Starta utvecklingsservern
npm run dev
```

2. Folderstruktur i ett Vite-projekt

När du öppnar mappen i din kodredigerare (t.ex. VS Code) kommer du att mötas av en standardiserad struktur. Här är vad de viktigaste delarna gör:

node_modules/	Här lagras alla färdiga bibliotek och paket som din applikation behöver (inklusive React). Denna mapp skapas automatiskt av <code>npm install</code> .
public/	Statiska filer som bilder, ikoner eller fonter som ska kopieras rakt av till den slutgiltiga webbplatsen utan att bearbetas.
src/	Hjärtat i ditt projekt! Det är här all din egen kod, dina komponenter, stilar och logik hamnar.
index.html	Projektets enda riktiga HTML-fil. Den ligger i roten och fungerar som "skalet" för hela din applikation.
package.json	Konfigurationsfil som håller koll på vilka beroenden projektet har samt vilka kommandon (scripts) du kan köra (t.ex. <code>npm run dev</code>).

3. Hur startar allt? (Resan från HTML till React)

Det kan ibland kännas förvirrande eftersom man sällan rör filer som `index.html` i vardaglig React-kod. Men så här hänger startkedjan ihop steg för steg:

Steg 1: index.html (Startpunkten)

När Vite startar letar den efter `index.html` i roten. Där finns en helt vanlig HTML-struktur med ett tomt div-element som har ett ID, samt en länk till start-skriptet:

```
<!doctype html>
<html lang="sv">
  <head>
    <meta charset="UTF-8" />
    <title>Vite + React</title>
  </head>
  <body>
    <!-- Här injiceras hela din React-applikation dynamiskt -->
    <div id="root"></div>

    <!-- Vite laddar in vår JavaScript/TypeScript-fil härifrån -->
    <script type="module" src="/src/main.tsx"></script>
  </body>
</html>
```

Steg 2: main.tsx (Bryggan till React)

Filen `src/main.tsx` är bryggan mellan den statiska HTML-filen och React. Den letar upp elementet med id `root` och säger åt React att "montera" (renderar ut) vår huvudkomponent där:

```
import React from 'react'
import ReactDOM from 'react-dom/client'
import App from './App.tsx'
import './index.css'

// Här kopplas React ihop med div-elementet "root" i index.html
ReactDOM.createRoot(document.getElementById('root')!).render(
  <React.StrictMode>
    <App />
  </React.StrictMode>,
)
```

Steg 3: App.tsx (Huvudkomponenten)

När `main.tsx` körs laddar den in `App.tsx`. Det är här din egen applikation börjar på riktigt. Det är en vanlig React-komponent som sedan kan innehålla hur många underkomponenter som helst (som `UserCard`, `Navbar`, etc.):

```
import { useState } from 'react'

export default function App() {
  const [count, setCount] = useState(0)

  return (
    <div style={{ padding: '2rem' }}>
      <h1>Min första React-app</h1>
      <button onClick={() => setCount(count + 1)}>
        Klickad {count} gånger
      </button>
    </div>
  )
}
```